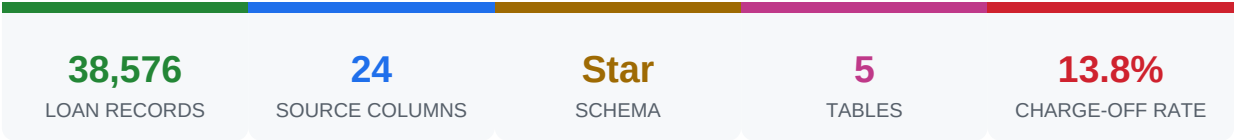


# BANK LOAN ANALYSIS

SQL Data Modelling & Validation Showcase

MySQL 8 · Star Schema · Power BI



This document covers the end-to-end SQL pipeline for the Bank Loan Analysis portfolio project — from raw CSV ingestion and star schema design to dimension table construction, surrogate key resolution, and post-load validation. Every design decision is documented with a clear rationale.

|         |             |          |      |                  |                  |
|---------|-------------|----------|------|------------------|------------------|
| MySQL 8 | Star Schema | Power BI | CTEs | Window Functions | Stored Procedure |
|---------|-------------|----------|------|------------------|------------------|

---

## 1. Project Overview

The source dataset is a single **financial\_loan.csv** file — 38,576 loan records across 24 columns. Rather than loading all columns into a flat table, every column was explicitly evaluated: three dropped, four normalised into dimensions, seventeen retained in the fact table.

| Item          | Detail                                              |
|---------------|-----------------------------------------------------|
| Database      | MySQL 8.x                                           |
| Source        | financial_loan.csv — 38,576 rows, 24 columns        |
| Schema        | Star schema — 1 fact table, 4 dimension tables      |
| BI layer      | Power BI (MySQL connector)                          |
| Date range    | Full calendar year 2021 — 365 days                  |
| Loan statuses | Fully Paid 83.3%   Charged Off 13.8%   Current 2.8% |

## 2. Column Accounting — All 24 Columns Decided

Every column in the source CSV was explicitly placed into one of three buckets. None were carried forward by default.

### Dropped (3)

| Column           | Type    | Reason                                                            |
|------------------|---------|-------------------------------------------------------------------|
| member_id        | INT     | Duplicates loan_id function — no independent business meaning     |
| application_type | VARCHAR | 100% single value (INDIVIDUAL) — zero variation or filter utility |
| emp_title        | VARCHAR | 28,524 unique free-text values — cannot be meaningfully grouped   |

### Moved to Dimensions (4)

| Column     | Destination | Rationale                                                    |
|------------|-------------|--------------------------------------------------------------|
| grade      | dim_grade   | 7 unique values — forms grade hierarchy with sub_grade       |
| sub_grade  | dim_grade   | 35 unique values — child of grade (A1–G5)                    |
| purpose    | dim_purpose | 14 values — enriched with category and data-driven risk tier |
| issue_date | dim_date    | Converted to date_id FK — full 365-day calendar required     |

### Retained in fact\_loan (17)

Low-cardinality text attributes (home\_ownership: 5 values, verification\_status: 3 values) stayed in the fact table. A dimension with the same row count as the fact table is not normalisation — it is duplication.

| Column              | Type          | Notes                                                     |
|---------------------|---------------|-----------------------------------------------------------|
| loan_id             | INT PK        | Primary key — renamed from id for clarity                 |
| loan_status         | VARCHAR       | Core outcome — every analysis resolves here               |
| loan_amount         | INT           | Total capital deployed                                    |
| int_rate            | DECIMAL(6,4)  | Stored as decimal: 0.1205 = 12.05%                        |
| installment         | DECIMAL(10,2) | Monthly payment burden                                    |
| dti                 | DECIMAL(6,4)  | Debt-to-income ratio                                      |
| total_payment       | DECIMAL(12,2) | Actual amount recovered from borrower                     |
| total_acc           | INT           | Breadth of borrower credit history                        |
| term                | VARCHAR(15)   | 36 or 60 months — TRIM applied (source has leading space) |
| home_ownership      | VARCHAR       | 5 unique values — kept in fact                            |
| verification_status | VARCHAR       | 3 unique values — kept in fact                            |
| emp_length          | VARCHAR       | 11 structured values — TRIM applied                       |
| annual_income       | DECIMAL(12,2) | Used for income band segmentation                         |
| last_payment_date   | DATE          | Gap vs issue_date indicates default timing                |

| Column                | Type   | Notes                                     |
|-----------------------|--------|-------------------------------------------|
| last_credit_pull_date | DATE   | Bank monitoring activity post-issuance    |
| next_payment_date     | DATE   | Populated even on Charged Off loans       |
| region_id             | INT FK | FK to dim_region — resolves address_state |

---

### 3. Star Schema Design

One central fact table, four dimension tables. All relationships are Many-to-One from fact to dimension — the correct direction for Power BI slicers and cross-filtering.

| Table       | Rows   | Purpose                                                       |
|-------------|--------|---------------------------------------------------------------|
| fact_loan   | 38,576 | Measures, outcomes, FK references                             |
| dim_grade   | 35     | Credit grade hierarchy A1–G5 with risk labels                 |
| dim_purpose | 14     | Loan purpose with category grouping and data-driven risk tier |
| dim_date    | 365    | Full 2021 calendar for Power BI time intelligence             |
| dim_region  | 50     | US state codes mapped to four Census Bureau regions           |

---

## 4. Dimension Tables

### 4.1 dim\_grade — Credit Grade Hierarchy

Grade and sub\_grade are dependent — A1 through A5 only exist within grade A. A compound ORDER BY on both columns assigns surrogate keys in clean A1-to-G5 sequence. The *risk\_label* enrichment column translates letter codes into business language for Power BI slicers — it did not exist in the source data.

```
CREATE TABLE dim_grade (  
    grade_id      INT PRIMARY KEY AUTO_INCREMENT,  
    grade         VARCHAR(5) NOT NULL,  
    sub_grade     VARCHAR(5) NOT NULL,  
    risk_label    VARCHAR(20) NOT NULL  
);  
  
INSERT INTO dim_grade (grade, sub_grade, risk_label)  
SELECT DISTINCT  
    grade,  
    TRIM(sub_grade),    -- source CSV has a leading space; omitting TRIM silently breaks JOINS  
    CASE grade  
        WHEN 'A' THEN 'Very Low Risk'  
        WHEN 'B' THEN 'Low Risk'  
        WHEN 'C' THEN 'Moderate Risk'  
        WHEN 'D' THEN 'Medium-High Risk'  
        WHEN 'E' THEN 'High Risk'  
        WHEN 'F' THEN 'Very High Risk'  
        WHEN 'G' THEN 'Extreme Risk'  
    END AS risk_label  
FROM stg_loans  
ORDER BY grade, TRIM(sub_grade); -- controls surrogate key sequence
```

### 4.2 dim\_purpose — Data-Driven Risk Classification

Risk tiers are derived from actual charge-off rates using a two-stage CTE — not hardcoded. Thresholds at 18% and 11% reflect natural breaks in the real data distribution. purpose\_category groups 14 raw purposes into 6 business-meaningful categories for slicing.

```

-- Stage 1: Calculate actual charge-off rate per purpose
WITH purpose_base AS (
    SELECT
        purpose,
        ROUND(
            100.0 * SUM(CASE WHEN loan_status = 'Charged Off' THEN 1 ELSE 0 END)
            / COUNT(*),
            2
        ) AS charge_off_pct
    FROM stg_loans
    GROUP BY purpose
),

-- Stage 2: Assign risk tier from data-driven thresholds
purpose_ranked AS (
    SELECT
        purpose,
        charge_off_pct,
        CASE
            WHEN charge_off_pct >= 18 THEN 'High Risk'
            WHEN charge_off_pct >= 11 THEN 'Medium Risk'
            ELSE 'Low Risk'
        END AS risk_tier
    FROM purpose_base
)

INSERT INTO dim_purpose (purpose, purpose_category, risk_tier)
SELECT
    pr.purpose,
    CASE
        WHEN pr.purpose = 'small business' THEN 'Business'
        WHEN pr.purpose IN ('car', 'major purchase') THEN 'Asset Purchase'
        WHEN pr.purpose IN ('credit card', 'Debt consolidation') THEN 'Debt Management'
        WHEN pr.purpose IN ('house', 'home improvement',
                           'renewable_energy') THEN 'Property'
        WHEN pr.purpose IN ('medical', 'moving', 'wedding') THEN 'Personal Need'
        ELSE 'Other'
    END AS purpose_category,
    pr.risk_tier
FROM purpose_ranked pr;

```

| Purpose            | Charge-Off % | Risk Tier   |
|--------------------|--------------|-------------|
| small_business     | 25.6%        | High Risk   |
| renewable_energy   | 18.1%        | High Risk   |
| educational        | 15.9%        | Medium Risk |
| house              | 15.6%        | Medium Risk |
| Debt consolidation | 14.6%        | Medium Risk |
| car                | 10.3%        | Low Risk    |
| credit_card        | 10.2%        | Low Risk    |
| wedding            | 9.3%         | Low Risk    |

### 4.3 dim\_date — Full Calendar via Stored Procedure

A full, unbroken 365-day calendar is required — not just the 65 unique issue dates that exist in the data. Power BI time intelligence functions (DATESMTD, DATEADD, SAMEPERIODLASTYEAR) misfire on sparse date tables. A stored procedure generates the complete 2021 calendar using a WHILE loop, then is dropped — the data persists, the generator does not.

```
CREATE TABLE dim_date (  
    date_id          INT PRIMARY KEY,      -- YYYYMMDD integer: easy to read when debugging JOINS  
    full_date        DATE                  NOT NULL,  
    day              INT                  NOT NULL,  
    month            INT                  NOT NULL,  
    month_name       VARCHAR(15) NOT NULL,  
    quarter          INT                  NOT NULL,  
    year             INT                  NOT NULL,  
    week_day_num     INT                  NOT NULL,  
    week_day_name    VARCHAR(15) NOT NULL,  
    is_weekend       TINYINT              NOT NULL,  
    month_sort_key   INT                  NOT NULL -- YEAR*100+MONTH: fixes Power BI alphabetical sort  
);  
  
DELIMITER $$  
CREATE PROCEDURE fill_dim_date()  
BEGIN  
    DECLARE curr_date DATE DEFAULT '2021-01-01';  
    DECLARE end_date  DATE DEFAULT '2021-12-31';  
  
    WHILE curr_date <= end_date DO  
        INSERT INTO dim_date VALUES (  
            DATE_FORMAT(curr_date, '%Y%m%d'),  
            curr_date,  
            DAY(curr_date),  
            MONTH(curr_date),  
            MONTHNAME(curr_date),  
            QUARTER(curr_date),  
            YEAR(curr_date),  
            DAYOFWEEK(curr_date),      -- MySQL: 1=Sunday, 7=Saturday (not Monday=1)  
            DAYNAME(curr_date),  
            CASE WHEN DAYOFWEEK(curr_date) IN (1, 7) THEN 1 ELSE 0 END,  
            YEAR(curr_date) * 100 + MONTH(curr_date)  
        );  
        SET curr_date = DATE_ADD(curr_date, INTERVAL 1 DAY);  
    END WHILE;  
END$$  
DELIMITER ;  
  
CALL fill_dim_date();  
DROP PROCEDURE fill_dim_date;
```

### 4.4 dim\_region — Geographic Grouping

Maps all 50 US state codes to four Census Bureau regions. Built with SELECT DISTINCT from staging — only states that actually appear in the data get a row. address\_state was deliberately *not* kept in the fact table; Power BI accesses it through the region\_id FK relationship, removing geographic redundancy.



```

CREATE TABLE dim_region (
  region_id INT AUTO_INCREMENT PRIMARY KEY,
  state_code VARCHAR(5) NOT NULL,
  region VARCHAR(20) NOT NULL
);

INSERT INTO dim_region (state_code, region)
SELECT DISTINCT
  address_state,
  CASE
    WHEN address_state IN ('CT', 'ME', 'MA', 'NH', 'RI', 'VT', 'NJ', 'NY', 'PA')
    THEN 'Northeast'
    WHEN address_state IN ('IL', 'IN', 'MI', 'OH', 'WI', 'IA', 'KS', 'MN', 'MO', 'NE', 'SD')
    THEN 'Midwest'
    WHEN address_state IN ('DE', 'FL', 'GA', 'MD', 'NC', 'SC', 'VA', 'DC', 'WV',
                          'AL', 'KY', 'MS', 'TN', 'AR', 'LA', 'OK', 'TX')
    THEN 'South'
    WHEN address_state IN ('AZ', 'CO', 'ID', 'MT', 'NV', 'NM', 'UT', 'WY',
                          'AK', 'CA', 'HI', 'OR', 'WA')
    THEN 'West'
    ELSE 'Other'
  END AS region
FROM stg_loans
ORDER BY address_state;

```

| Region    | Loans  | Share |
|-----------|--------|-------|
| South     | 12,651 | 32.8% |
| West      | 11,067 | 28.7% |
| Northeast | 9,459  | 24.5% |
| Midwest   | 5,399  | 14.0% |

---

## 5. fact\_loan — Central Fact Table

38,576 rows — one per loan. The INSERT SELECT resolves four surrogate keys simultaneously, replacing all raw text values from staging with integer IDs. dim\_region uses LEFT JOIN so a state mismatch inserts a NULL region\_id rather than silently dropping the row. STR\_TO\_DATE handles dates stored as DD-MM-YYYY in the source.

```
CREATE TABLE fact_loan (  
  loan_id          INT PRIMARY KEY,  
  grade_id         INT          NOT NULL,  
  purpose_id       INT          NOT NULL,  
  date_id          INT          NOT NULL,  
  region_id        INT          NOT NULL,  
  loan_status      VARCHAR(20)  NOT NULL,  
  loan_amount      INT          NOT NULL,  
  int_rate         DECIMAL(6,4) NOT NULL,  
  installment      DECIMAL(10,2) NOT NULL,  
  dti              DECIMAL(6,4) NOT NULL,  
  total_acc        INT          NOT NULL,  
  total_payment    DECIMAL(12,2) NOT NULL,  
  term             VARCHAR(15)  NOT NULL,  
  home_ownership   VARCHAR(20)  NOT NULL,  
  verification_status VARCHAR(20) NOT NULL,  
  emp_length       VARCHAR(20),  
  annual_income    DECIMAL(12,2),  
  last_payment_date DATE,  
  last_credit_pull_date DATE,  
  next_payment_date DATE,  
  FOREIGN KEY (grade_id) REFERENCES dim_grade(grade_id),  
  FOREIGN KEY (purpose_id) REFERENCES dim_purpose(purpose_id),  
  FOREIGN KEY (date_id) REFERENCES dim_date(date_id),  
  FOREIGN KEY (region_id) REFERENCES dim_region(region_id)  
);
```

```

INSERT INTO fact_loan (
    loan_id, grade_id, purpose_id, date_id, region_id,
    loan_status, loan_amount, int_rate, installment, dti,
    total_acc, total_payment, term, home_ownership,
    verification_status, emp_length, annual_income,
    last_payment_date, last_credit_pull_date, next_payment_date
)
SELECT
    s.id,
    g.grade_id,
    p.purpose_id,
    DATE_FORMAT(STR_TO_DATE(s.issue_date, '%d-%m-%Y'), '%Y%m%d'),
    r.region_id,
    s.loan_status,
    s.loan_amount,
    s.int_rate,
    s.installment,
    s.dti,
    s.total_acc,
    s.total_payment,
    TRIM(s.term),
    s.home_ownership,
    s.verification_status,
    TRIM(s.emp_length),
    s.annual_income,
    STR_TO_DATE(s.last_payment_date, '%d-%m-%Y'),
    STR_TO_DATE(s.last_credit_pull_date, '%d-%m-%Y'),
    STR_TO_DATE(s.next_payment_date, '%d-%m-%Y')
FROM stg_loans s
JOIN    dim_grade    g ON g.grade      = s.grade
        AND g.sub_grade = TRIM(s.sub_grade) -- compound: A1 and B1 are both '1'
JOIN    dim_purpose    p ON p.purpose      = s.purpose
JOIN    dim_date      d ON d.full_date = STR_TO_DATE(s.issue_date, '%d-%m-%Y')
LEFT JOIN dim_region  r ON r.state_code = s.address_state; -- LEFT: never drop a row

```

## 6. Validation & Metrics Queries

Two layers of validation were run after loading. First, structural checks confirm row counts and FK integrity. Second, KPI queries establish SQL baselines that every Power BI DAX measure was cross-validated against before publishing. If DAX and SQL disagreed, DAX was fixed.

### 6.1 Post-Load Structural Checks

```
-- Row count must match staging exactly
SELECT COUNT(*) AS fact_row_count
FROM fact_loan;
-- Expected: 38,576

-- Zero unmatched FKs means no rows were silently dropped during INSERT
SELECT COUNT(*) AS orphaned_grades
FROM fact_loan f
LEFT JOIN dim_grade g ON g.grade_id = f.grade_id
WHERE g.grade_id IS NULL;

SELECT COUNT(*) AS orphaned_purposes
FROM fact_loan f
LEFT JOIN dim_purpose p ON p.purpose_id = f.purpose_id
WHERE p.purpose_id IS NULL;

SELECT COUNT(*) AS orphaned_dates
FROM fact_loan f
LEFT JOIN dim_date d ON d.date_id = f.date_id
WHERE d.date_id IS NULL;

-- Catch silent date failures (0000-00-00 loads as NULL if STR_TO_DATE format is wrong)
SELECT
    SUM(CASE WHEN last_payment_date IS NULL THEN 1 ELSE 0 END) AS null_payment_dates,
    SUM(CASE WHEN last_credit_pull_date IS NULL THEN 1 ELSE 0 END) AS null_credit_dates,
    SUM(CASE WHEN next_payment_date IS NULL THEN 1 ELSE 0 END) AS null_next_dates
FROM fact_loan;
```

### 6.2 KPI Baseline — SQL Ground Truth

```
SELECT
    COUNT(*) AS total_loans,
    SUM(loan_amount) AS total_funded,
    ROUND(SUM(total_payment)) AS total_received,
    ROUND(AVG(int_rate) * 100, 2) AS avg_int_rate_pct,
    ROUND(AVG(dti) * 100, 2) AS avg_dti_pct,
    SUM(CASE WHEN loan_status = 'Charged Off' THEN 1 ELSE 0 END) AS bad_loans,
    ROUND(
        100.0 * SUM(CASE WHEN loan_status = 'Charged Off' THEN 1 ELSE 0 END)
        / COUNT(*),
        1
    ) AS bad_loan_pct
FROM fact_loan;
```

| Metric           | Expected Value |
|------------------|----------------|
| total_loans      | 38,576         |
| total_funded     | \$435,757,075  |
| total_received   | \$473,070,933  |
| avg_int_rate_pct | 12.05%         |
| avg_dti_pct      | 13.33%         |
| bad_loans        | 5,333          |
| bad_loan_pct     | 13.8%          |

## 6.3 Month-over-Month Application Growth

Three-stage CTE: aggregate by month, apply LAG() to get prior period, then compute absolute and percentage change. CASE guard handles the first month (no prior row) and prevents division by zero.

```
WITH monthly_data AS (  
    SELECT  
        d.year,  
        d.month,  
        DATE_FORMAT(d.full_date, '%Y-%m') AS month_no,  
        COUNT(f.loan_id) AS current_month_applications  
    FROM fact_loan f  
    JOIN dim_date d ON f.date_id = d.date_id  
    GROUP BY d.year, d.month, month_no  
,  
  
    prior_month AS (  
        SELECT  
            month_no,  
            current_month_applications,  
            LAG(current_month_applications) OVER (ORDER BY year, month) AS prev_applications  
        FROM monthly_data  
    ),  
  
    mom_calc AS (  
        SELECT  
            month_no,  
            current_month_applications,  
            prev_applications,  
            current_month_applications - prev_applications AS mom_change  
        FROM prior_month  
    )  
  
    SELECT  
        month_no,  
        current_month_applications,  
        prev_applications,  
        mom_change,  
        CASE  
            WHEN prev_applications IS NULL OR prev_applications = 0 THEN NULL  
            ELSE ROUND((mom_change / prev_applications) * 100.0, 2)  
        END AS mom_pct_diff  
    FROM mom_calc  
    ORDER BY month_no;
```

## 6.4 Good Loan vs Bad Loan Split

Single-pass conditional aggregate — counts all status categories in one table scan, avoiding subqueries or self-joins.

```

WITH loan_counts AS (
    SELECT
        COUNT(loan_id) AS total_applications,
        SUM(CASE WHEN loan_status IN ('Fully Paid', 'Current') THEN 1 ELSE 0 END) AS good_loans,
        SUM(CASE WHEN loan_status = 'Charged Off' THEN 1 ELSE 0 END) AS bad_loans
    FROM fact_loan
)
SELECT
    total_applications,
    good_loans,
    ROUND((good_loans / total_applications) * 100.0, 2) AS good_loan_pct,
    bad_loans,
    ROUND((bad_loans / total_applications) * 100.0, 2) AS bad_loan_pct
FROM loan_counts;

```

## 6.5 Charge-Off Rate by Credit Grade

```

SELECT
    g.grade,
    COUNT(*) AS total_loans,
    SUM(CASE WHEN f.loan_status = 'Charged Off' THEN 1 ELSE 0 END) AS charged_off,
    ROUND(
        100.0 * SUM(CASE WHEN f.loan_status = 'Charged Off' THEN 1 ELSE 0 END)
        / COUNT(*),
        2
    ) AS charge_off_pct
FROM fact_loan f
JOIN dim_grade g ON f.grade_id = g.grade_id
GROUP BY g.grade
ORDER BY g.grade DESC;

```

## 6.6 Verification Status Distribution

SUM(COUNT(\*)) OVER () computes the grand total in the same pass — no subquery or self-join needed.

```

SELECT
    verification_status,
    COUNT(*) AS total_loans,
    ROUND((COUNT(*) / SUM(COUNT(*) OVER ())) * 100.0, 2) AS pct_share
FROM fact_loan
GROUP BY verification_status
ORDER BY pct_share DESC;

```

## 6.7 Monthly Applications & Charge-Off Rate Over Time

```
WITH monthly_data AS (  
    SELECT  
        d.year,  
        d.month,  
        DATE_FORMAT(d.full_date, '%Y-%m') AS month_label,  
        COUNT(f.loan_id) AS total_applications,  
        SUM(CASE WHEN f.loan_status = 'Charged Off' THEN 1 ELSE 0 END) AS charged_off  
    FROM fact_loan f  
    JOIN dim_date d ON f.date_id = d.date_id  
    GROUP BY d.year, d.month, month_label  
)  
SELECT  
    month_label,  
    total_applications,  
    ROUND((charged_off / total_applications) * 100.0, 2) AS charge_off_rate  
FROM monthly_data  
ORDER BY month_label;
```

## 6.8 Charge-Off Rate by Loan Purpose

```
SELECT  
    p.purpose,  
    COUNT(*) AS total_loans,  
    ROUND(  
        100.0 * SUM(CASE WHEN l.loan_status = 'Charged Off' THEN 1 ELSE 0 END)  
        / COUNT(*),  
        2  
    ) AS charge_off_pct  
FROM fact_loan l  
LEFT JOIN dim_purpose p ON l.purpose_id = p.purpose_id  
GROUP BY p.purpose  
ORDER BY charge_off_pct DESC;
```

## 6.9 Recovery Analysis — Charged-Off Loans

NULLIF(SUM(loan\_amount), 0) prevents division by zero. net\_loss = funded minus recovered. recovery\_rate shows how much of bad loan principal came back.



```

WITH charged_off AS (
    SELECT
        f.loan_amount,
        f.total_payment,
        r.region
    FROM fact_loan f
    JOIN dim_region r ON f.region_id = r.region_id
    WHERE f.loan_status = 'Charged Off'
)
SELECT
    COUNT(*) AS total_charged_off_loans,
    SUM(loan_amount) AS bad_loan_funded,
    SUM(total_payment) AS bad_loan_received,
    SUM(loan_amount) - SUM(total_payment) AS net_loss,
    ROUND(
        100.0 * SUM(total_payment) / NULLIF(SUM(loan_amount), 0),
        2
    ) AS recovery_rate_pct
FROM charged_off;

```

---

## 7. Key Design Decisions

### 1. Dynamic risk tiers, not hardcoded values

Most approaches manually assign risk labels. Here, tiers are computed from actual charge-off rates via a two-stage CTE. Thresholds at 18% and 11% reflect natural breaks in the real distribution — not arbitrary constants.

### 2. dim\_borrower rejected on granularity grounds

A borrower dimension was evaluated and dropped: one row per loan equals the fact table's row count, making it a denormalised copy rather than a true dimension. Low-cardinality attributes stay directly in fact\_loan.

### 3. Full 365-day calendar over a sparse date list

Loading only the 65 unique issue dates that exist in the data would silently break Power BI time intelligence (DATESMTD, DATEADD, SAMEPERIODLASTYEAR). A stored procedure generates the complete, unbroken 2021 calendar.

### 4. address\_state in dim\_region, not in fact\_loan

Power BI reaches state codes through the region\_id FK — no redundant column in the fact table. The filled map visual works through the relationship path.

### 5. grade and purpose are independent risk dimensions

A Grade A borrower taking a small business loan has an 11.4% charge-off rate; a wedding loan has 1.7%. Same credit quality, different use-of-funds risk. This interaction is only visible because both dimensions are in the model.